# ADVANCED STRING-BASED REGISTRATION RECORD ANALYZER

### Python Project Report

## Project Overview

This project reads structured registration strings, separates their fields, checks field lengths and formats, extracts registration details, detects duplicate registration IDs, and reports error classifications.

## Example record format

```
REG1001|Anu Kumar|anu@example.com|9876543210
```

Fields: Registration ID | Name | Email | Phone

Assumed validation rules for this example: ID must look like REG followed by four digits; name length must be 2–30 characters; email must have a basic valid format; phone must contain exactly 10 digits; every ID can appear only once.

## 1. Problem Statement

Develop a Python program that parses structured registration strings, validates the number of fields and their lengths/formats, extracts the required components, detects duplicate registration IDs, and classifies errors.

## 2. Objectives

• Split each registration string into its individual fields.

• Validate registration ID, name length, email, and phone number.

• Extract registration ID, name, email, and phone.

• Detect duplicate registration IDs.

• Classify invalid records by error type and display the results.

## 3. Assumptions and Validation Rules

• Fields are separated by the pipe symbol (|).

• A record must contain exactly four fields: ID, name, email, and phone.

• Registration ID format: REG followed by four digits, such as REG1001.

• Name must contain 2 to 30 characters.

• Email must contain a basic local part, @ symbol, and domain.

• Phone number must contain exactly 10 digits.

• A registration ID must not be repeated.

## 4. Algorithm

1. Start.

2. Read the list of registration strings.

3. Create an empty set to store registration IDs already seen.

4. Split each record using the | separator.

5. If the field count is not four, report FIELD_COUNT.

6. Validate ID, name length, email format, and phone format.

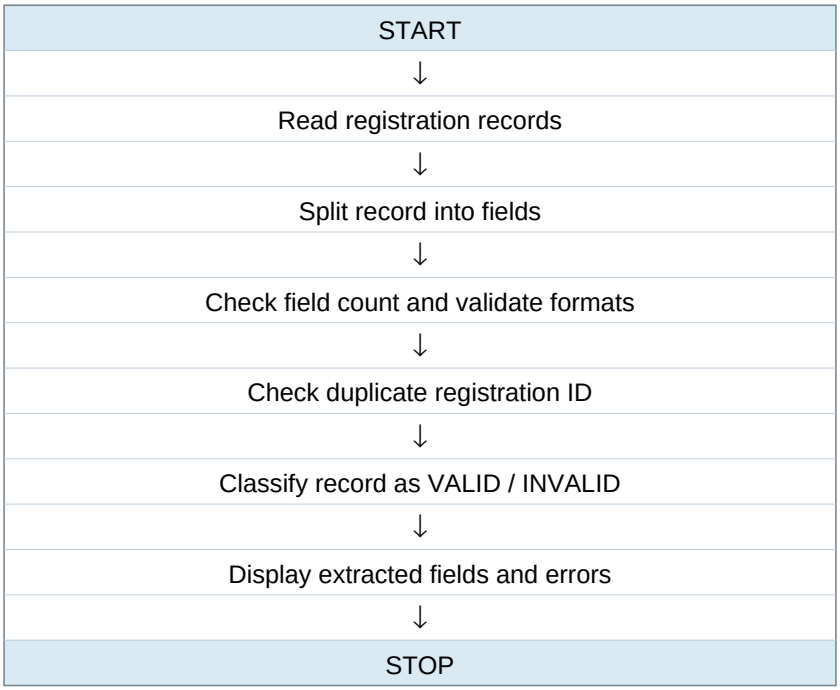
7. Check whether the ID already exists in the set; if so, report DUPLICATE_ID.

8. Mark the record VALID if no errors exist; otherwise mark it INVALID.

9. Display extracted fields, status, and error classifications.

10. Stop.

## 5. Flowchart




## 6. Pseudocode

```
BEGIN
    INPUT registration records
    seen_ids = empty set
    FOR each record:
        split record by "|"
        IF field count is not 4:
            error = FIELD_COUNT
        ELSE:
            extract ID, name, email, phone
            validate ID format and name length
            validate email and phone formats
            IF ID already in seen_ids:
                add DUPLICATE_ID error
            ELSE:
                add ID to seen_ids
            IF no errors: status = VALID
            ELSE: status = INVALID
        DISPLAY status and error classifications
    DISPLAY total, valid, and invalid counts
END
```

## 7. Python Source Code

Save as **advanced_string_registration_analyzer.py**.

```
# Advanced String-Based Registration Record Analyzer
import re

def analyze_records(records):
    seen_ids = set()
    results = []

    for record in records:
        errors = []
        parts = record.split("|")

        # Expected format: ID|NAME|EMAIL|PHONE
        if len(parts) != 4:
            results.append({
                "record": record,
                "status": "INVALID",
                "errors": ["FIELD_COUNT"]
            })
            continue

        reg_id, name, email, phone = [part.strip() for part in parts]

        if not (re.fullmatch(r"REG\d{4}", reg_id)):
            errors.append("INVALID_ID_FORMAT")
        if not (2 <= len(name) <= 30):
            errors.append("INVALID_NAME_LENGTH")
        if not re.fullmatch(r"[^@\s]+@[^@\s]+\.[^@\s]+", email):
            errors.append("INVALID_EMAIL_FORMAT")
        if not re.fullmatch(r"\d{10}", phone):
            errors.append("INVALID_PHONE_FORMAT")

        if reg_id in seen_ids:
            errors.append("DUPLICATE_ID")
        else:
            seen_ids.add(reg_id)

        status = "VALID" if not errors else "INVALID"
        results.append({
            "record": record,
            "id": reg_id,
            "name": name,
            "email": email,
            "phone": phone,
            "status": status,
```

## 7. Python Source Code (continued)

```
            "errors": errors if errors else ["NONE"]
        })

    return results


def main():
    records = [
        "REG1001|Anu Kumar|anu@example.com|9876543210",
        "REG1002|Ravi|ravi@example.com|9123456780",
        "REG1001|Sara Devi|sara@example.com|9988776655",
        "REG1004|K|bad-email|12345",
        "REG1005|Mohan Das|mohan@example.com|9000011111",
        "REG1006|Priya Sharma|priya@example.com"
    ]

    results = analyze_records(records)

    print("=" * 92)
    print("              ADVANCED STRING-BASED REGISTRATION RECORD ANALYZER")
    print("=" * 92)
    print(f"{'No.':<5}{'Registration ID':<18}{'Name':<18}{'Status':<12}Error Classification")
    print("-" * 92)

    for i, item in enumerate(results, 1):
        reg_id = item.get("id", "N/A")
        name = item.get("name", "N/A")
        errors = ", ".join(item["errors"])
        print(f"{i:<5}{reg_id:<18}{name:<18}{item['status']:<12}{errors}")

    print("-" * 92)
    valid_count = sum(1 for item in results if item["status"] == "VALID")
    invalid_count = len(results) - valid_count
    print(f"Total Records: {len(results)} | Valid: {valid_count} | Invalid: {invalid_count}")
    print("Extracted fields: Registration ID, Name, Email, Phone")
    print("Analysis completed successfully.")

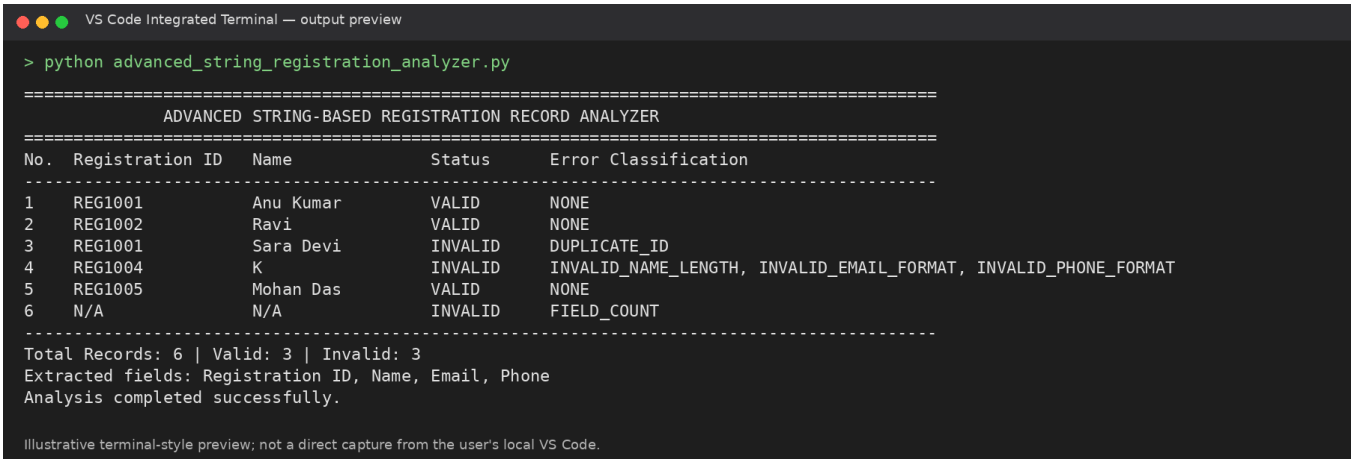

if __name__ == "__main__":
    main()
```

## 8. Test Cases

| Test | Input / condition | Expected classification |
|---|---|---|
| Valid record | Correct four fields and valid formats | VALID / NONE |
| Duplicate ID | Registration ID used earlier | INVALID / DUPLICATE_ID |
| Invalid name | Name has fewer than 2 characters | INVALID / INVALID_NAME_LENGTH |
| Invalid email | Email lacks a valid basic structure | INVALID / INVALID_EMAIL_FORMAT |
| Invalid phone | Phone has fewer or more than 10 digits | INVALID / INVALID_PHONE_FORMAT |
| Missing field | Record has fewer than four fields | INVALID / FIELD_COUNT |

## 9. Output Screenshot

This is an illustrative terminal-style preview, not a direct screenshot captured from the user's local VS Code application.




For a genuine screenshot, open the Python file in VS Code, choose Terminal → New Terminal, run **python advanced_string_registration_analyzer.py**, and capture the editor and terminal on your computer.

## 10. Conclusion

The program parses registration strings, validates field count and formats, extracts details, detects duplicate IDs, and displays clear error classifications. This helps identify incorrect or repeated registration records efficiently.